

```
# Program to demonstrate Left Shift and Right Shift operations
```

```
number = 40
```

```
shift_positions = 2
```

```
print(f"Original Number: {number} (Binary: {bin(number)[2:]})")
```

```
print(f"Positions to shift: {shift_positions}\n")
```

```
# 1. Left Shift Operation (<<)
```

```
left_shift_result = number << shift_positions
```

```
print("--- Left Shift (<<) ---")
```

```
print(f"Operation: {number} << {shift_positions}")
```

```
print(f"Result (Decimal): {left_shift_result}")
```

```
print(f"Result (Binary) : {bin(left_shift_result)[2:]})")
```

```
print(f"Explanation    : Multiplied by 2^{shift_positions} ({2**shift_positions})\n")
```

```
# 2. Right Shift Operation (>>)
```

```
number // (2 ** shift_positions)
```

```
right_shift_result = number >> shift_positions
```

```
print("--- Right Shift (>>) ---")
```

```
print(f"Operation: {number} >> {shift_positions}")
```

```
print(f"Result (Decimal): {right_shift_result}")
```

```
print(f"Result (Binary) : {bin(right_shift_result)[2:]})")
```

```
print(f"Explanation    : Integer divided by 2^{shift_positions} ({2**shift_positions})")
```

Output:

Original Number: 40 (Binary: 101000)

Positions to shift: 2

--- Left Shift (<<) ---

Operation: 40 << 2

Result (Decimal): 160

Result (Binary) : 10101000

Explanation : Multiplied by 2² (4)

--- Right Shift (>>) ---

Operation: 40 >> 2

Result (Decimal): 10

Result (Binary) : 1010

Explanation : Integer divided by 2² (4)